Signal Object Detection

Bulacu Daria

Group: 241

Faculty of Mathematics and Computer Science

University of Bucharest

2025-2026

1. First approach

1.1 Motivation

The first approach that I had but after consideration I concluded that it will not raise a decent accuracy no matter the tuning, was based on classical ML feature description in image processing [1]. Since we are dealing with radio spectrograms, I considered that one of the best structural descriptors would be HOG (Histogram of oriented gradients) for it captures the distribution of gradient orientations in an image region, making it useful for detecting objects with varying shapes and textures. By extracting hand-crafted features from each image and feeding them to a classical classifier I thought that the result would be a promising one and I could avoid training a deep learning model.

It was set to fail from the beginning since spectrograms by definition have a lot of static noise around the bandwidth and the most important observation from the image classification data set is that the local edge orientations were the same in some cases and only the spatial pattern across columns differed. The problem with the feature descriptor is that those specific patterns were destroyed. Because of that most images were misclassified by the RBF SVM model that I implemented at first and the accuracy dropped significantly. For that I did not consider looking forward into it so I switched to researching deep-learning models.

Before abandoning the classical route entirely, however, I made sure the conclusion was not premature. The approach was based on building a small, reusable pipeline. The first step was preprocessing each RGBA image, since the dataset is coloured with the viridis colormap specific to the format of a radio signal, so there were 4 channels with a constant alpha. Because of that I had to convert them to a single grayscale luminance channel and resize to a fixed size before any descriptor was computed. The conversion helps to reduce the numbers that the classifier has to work on, without throwing away any relevant information, because luminance preserves the underlying signal magnitude while reducing the channels to a single one.

1.2 Using a SVM Classifier

The classifier I built around these features is a Support Vector Machine, and it is worth explaining why it is the right classical tool here. An SVM is a supervised classifier that, instead of memorising examples, searches for the decision boundary that separates the classes with the largest possible margin.

Real feature spaces are rarely linearly separable, and mine certainly was not, so the SVM handles this through the kernel trick, so by using the RBF kernel the model can build curved, non-linear decision regions which respects the demands of the 5 visually overlapping classes.

1.3 Why an SVM rather than a distance-based classifier

I deliberately chose an SVM rather than a lazy, distance-based classifier such as k-Nearest-Neighbours, and the reasoning is rooted in the nature of the data. Because it doesn't do any real training, only by grouping the pictures together by a majority vote, the simplicity becomes an indispensable problem because of the curse of dimensionality, the sensitivity to the noise that fills the spectrogram around the bandwidth and the no notion of a generalizing margin. The SVM avoids all three and for a high-dimensional, noisy descriptor like HOG, the maximum-margin SVM is the better founded classical approach implementation.

1.4 Hyperparameter tuning and its analysis

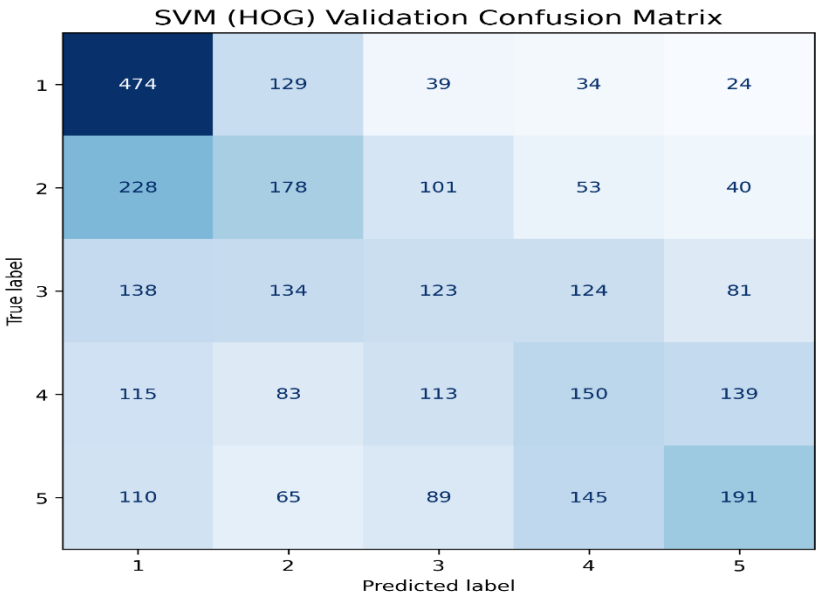
The SVM's two hyperparameters were chosen by grid search with three-fold stratified cross-validation. Because the RBF SVM scales roughly with the square of the number of training points, I ran the search on a stratified 4,000-sample subset and then refit the chosen configuration on the full training set. This keeps the grid search fast while letting the final model see all the data.

Hyperparameter	Values	Best
C	1, 10, 100	10
gamma	Scale, 0.001, 0.01	Scale

The outcome points to the conclusion of the data not supporting a complex decision boundary, and the cross-validation scores across the entire grid were clustered tightly in the low 0.30s, meaning that no combination of hyperparameters can raise the accuracy making this a limitation of the HOG features, rather than the classifier.

1.5 Confusion Matrix

The confusion matrix clearly tells us the error of our ways. Instead of a strong diagonal with a single confusable class pair, it shows errors spread broadly across all five classes.



(Figure 1: SVM Confusion Matrix)

2. Second Approach

Having accepted that the features were the bottleneck, I let the model learn its own features. I built a small VGG-style convolutional neural network in Keras using the Functional API, which lets me express the forward pass as a chain of layers and gives me freedom to insert components such as the augmentation block at the front of the graph.

3.1 Preprocessing and label handling

The images go through the same grayscale conversion as before, again, because vivardis is one-dimensional, so one luminance channel keeps the magnitude and spares the network from learning four times as many input numbers, of course then followed by normalisation from [0, 255] to [0, 1], because gradient descent behaves badly when the inputs are large and unscaled. The labels needed one careful adjustment, which resulted from a very intense debugging session: the dataset labels run from 1 to 5, but a softmax layer with five neurons is zero-indexed, so the loss function expects classes 0-4. I therefore subtract one from every label before training and add one back when writing the submission("minus one to learn, plus one to submit"). Forgetting that +1 at submission time was, in fact, a mistake I made early on: the model was healthy but the predictions were shifted by one class, and the leaderboard score collapsed to about 0.08 (below chance) until I traced it back to the missing offset.

3.2 Architecture

The network is organised as three convolutional blocks of increasing width, followed by a compact classifier head. Each block contains two 3×3 convolutions, each followed by batch normalisation and a ReLU activation, and then a 2×2 max-pooling layer and a dropout layer. I use two convolutions per block rather than one because two stacked 3×3 filters cover a larger effective receptive field while using fewer parameters than a single large filter, and they add an extra non-linearity, which makes this the VGG insight [2]. Padding is kept same so that the convolutions never change the spatial size; only the pooling layers do, which keeps the architecture predictable as the spatial resolution halves from block to block while the channel depth grows. Batch normalisation stabilises training and allows a higher learning rate; ReLU provides the non-linearity without which the stacked layers would collapse into a single linear function; max-pooling gives translation tolerance; and dropout (lighter, 0.25, in the convolutional blocks and heavier, 0.5, on the dense head) fights overfitting. After the last block I use global average pooling instead of a flatten-plus-dense, because flattening an 8×8×128 volume would create a huge, overfitting-prone dense layer, whereas global average pooling collapses each feature map to a single number and produces a compact 128-vector. The head is a dense layer with batch normalisation, ReLU and dropout, ending in a five-neuron softmax.

The model is compiled with sparse categorical cross-entropy, the integer-label form of cross-entropy, matching my 0-4 labels, optimised with Adam, and tracked with accuracy as the metric (the loss is what is minimised; accuracy is only reported). Training uses three callbacks every time:

1. Early stopping so that we don't memorize the data, resulting in overfitting and have a big drop on the validation set. It monitors validation accuracy and restores the best weights throwing away the overfitted weights from the final epoch;
2. Learning-rate reduction on plateau steps in before EarlyStopping, in case of a high learning rate, by cutting it in half after stalling for 5 epochs. This happens usually when we get close to the optimal solution and it starts taking very violent back and forth steps, and that way we can fix it by just taking smaller steps to find a better minimum;
3. Model checkpoint that keeps the best weights on disk, since my first running of the code was local and in case of the set-up breaking down, I could restores it from the disk where the physical .keras file lies.

3.3 Data augmentation

To regularise the network I apply small, label-preserving transformations to the training images: a random translation and a random contrast adjustment. I deliberately placed the augmentation inside the model, as its first layer, rather than in the input pipeline. This has two advantages: it runs on the GPU as part of the forward pass, and Keras automatically activates it only during training (it passes the data through unchanged at inference), so there is no risk of accidentally augmenting the validation or test set.

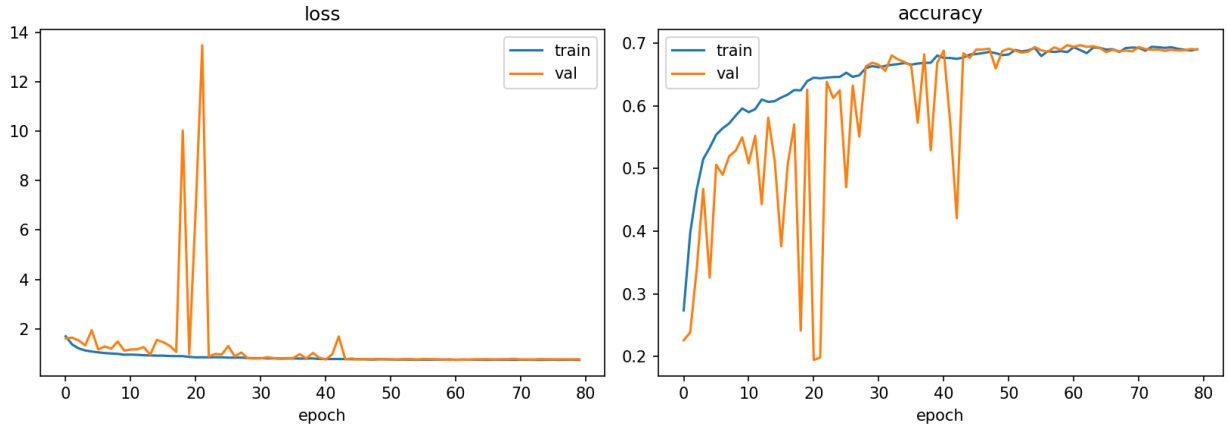
3.4 The evolution of the configuration

The initial configuration used convolutional filters of (32, 64, 128), input images resized to 64×64, and a RandomRotation augmentation layer. This attempt had a better accuracy than the classic classifier, but still not satisfactory enough.

I looked into the image characteristics and by compacting the height, it throws away half of its resolution and distorts the aspect ratio. Because the classes are distinguished by thin line structures across the frequency axis, halving its resolution was erasing the distinguishable information. Switching to 126x64 preserves the large axis and only stretches the time one a little. That way there was a trade-off between computation and accuracy, by feeding the CNN more numbers.

Augmentation must be label-preserving, and by using the RandomRotation the set of near-vertical lines structures were tilted and therefore distorting the feature that defines a class.

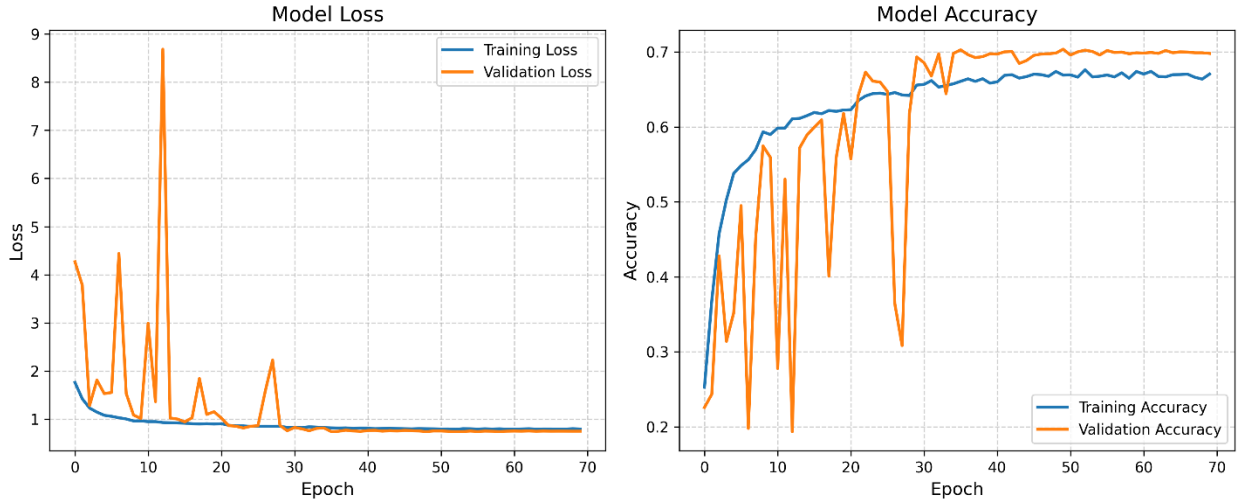
By fixing the augmentation problem, the network had more genuine detail to work with so I widened the blocks, by doubling the filters so that it can receive the capacity of capturing finer patterns.



(Figure 2: Initial Configuration Cnn)

The training curve is well-behaved in both panels, and the validation one spikes until about the 45th epoch, finally both curves sitting on top of each other and converging. This doesn't necesarilly mean that the accuracy is higher, but that it converges to a worse result smoothly.

Because of the training of the first attempt model in a more gentler regime – lower capacity, heavy regularization we result in a more unconfident model.



(Figure 3: Final Configuration CNN)

The spiking of the validation line that can be seen between epochs 0-35 is based on the high initial learning rate, but by using the ReduceLROnPlateau we chopped the learning rate, locking in with the training line.

We can also observe that the validation accuracy is way higher than the training one. This reversion happens because of the dropout (during training it shuts neurons off, but on validation dropout is turned off allowing it to use its full capacity). Also because of the changes brough on the images in the augmentation part, because the validation images are way easier to interpret, we have a crescendo movement in validation acc.

By training in a more harsh regime we can say confidently that the predictions are made with more confidence than the first attempt.

3.5 Hyperparameter experiments

To satisfy the hyperparameter-tuning requirement and to understand the model's sensitivity, I trained several deliberately-altered variants of the network, changing one thing at a time and recording each one's best validation accuracy. Resetting the same random seed before each run

makes the comparison fair, so that any difference is attributable to the hyperparameter and not to a luckier initialisation.

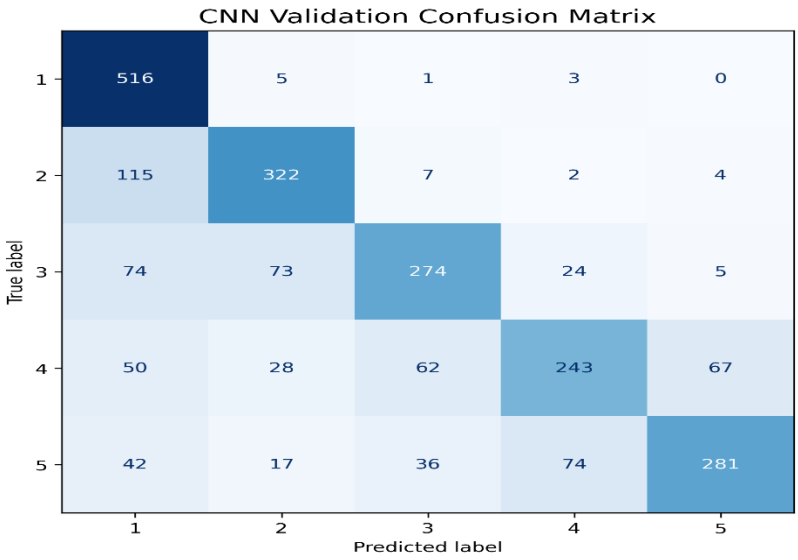
Variant	Change relative to baseline	Purpose
baseline	learning rate 1e-3, Adam	reference
higher_lr	learning rate 1e-2	test a 10× larger step
lower_lr	learning rate 1e-4	test a 10× smaller step
sgd_momentum	optimiser SGD (momentum 0.9)	compare optimisers
no_dropout	dropout 0.0 everywhere	demonstrate overfitting
smaller_model	filters (16, 32, 64)	a lighter network

Variant	Validation Acc	Learning Rate	Optimiser
baseline	0.703656	0.0010	adam
sgd_momentum	0.455054	0.0100	
no_dropout	0.424516	0.0010	adam
higher_lr	0.404731	0.0100	adam
smaller_model	0.311828	0.0010	adam
lower_lr	0.258925	0.0001	adam

These experiments confirmed that the moderate 1e-3 learning rate with Adam was the best general choice, that removing dropout widened the train–validation gap as expected, and that the smaller network underperformed the wider one — consistent with the capacity argument behind my filter change.

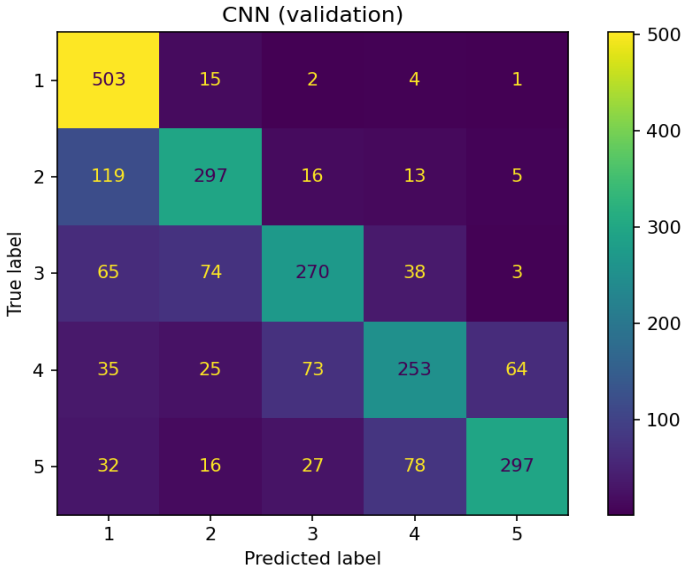
3.6 Confusion matrix

Unlike the classical model's broadly-spread errors, the CNN's mistakes are concentrated on a small number of genuinely similar class pairs, with a strong diagonal — the signature of a model that has actually learned the distinguishing structure.



(Figure 4: Confusion Matrix CNN Final)

We can also state that there's a visible difference between the initial configuration before modifying the regime the model makes predictions in and the final one.(I initially used the viridis colormap, but then changed it to a sequential-single hue scale for representing the count magnitude)



(Figure 5: Initial Confusion Matrix Config)

3.7 Ensembling

A final attempt on raising the accuracy was based on averaging the predictions of several CNN's. I based this on two strategies : seed ensemble and k-fold. For the first one I train the same

architecture several times with different random seeds and average their softmax outputs. The different initialisations give us the guarantee that the models will make partly-independent errors so by averaging them using soft voting, we can cancel some of the noise. The second one, which was stronger, used splitting of the data set into 5 folds and train each model on one fold and validating on the fold that it did not train on. That way we are given a pretty good cross-validation accuracy from the out of fold predictions and using the same logic as the seed ensemble, we average the predictions. This way we don't have data leakage.

After independently implementing these two methods, I considered the possibility of combining so that these 5 folds can have their models be recursively trained with the seed ensembling. I did not go through with it because of the deadline nearing up, so there's no further consideration of this method.